```
#include <iostream>
#include <queue>
#include <stack>
#include <fstream>
#include <string>

using namespace std;

struct Vehicle
{
   string license;
   string type;
};

struct Record
{
   string license;
   int fineCount;
   string status;
   string startDate;
   string endDate;
   int fineAmount;
};

queue<Vehicle> lane[4];

string lights[4] = {"RED", "RED", "RED", "RED"};
```

```
stack<string> lightStack;

Record records[100];

int recordCount = 0;

// ================= LOAD FILE =================
void loadFile()
{
   ifstream file("violations.txt");

   while (file >> records[recordCount].license
            >> records[recordCount].fineCount
            >> records[recordCount].status
            >> records[recordCount].startDate
            >> records[recordCount].endDate
            >> records[recordCount].fineAmount)
   {
      recordCount++;
   }

   file.close();
}

// ================= SAVE FILE =================
void saveFile()
{
   ofstream file("violations.txt");
```

```
    for (int i = 0; i < recordCount; i++)
    {
        file << records[i].license << " "
             << records[i].fineCount << " "
             << records[i].status << " "
             << records[i].startDate << " "
             << records[i].endDate << " "
             << records[i].fineAmount << endl;
    }

    file.close();
}

// ================= FIND RECORD =================
int findRecord(string license)
{
    for (int i = 0; i < recordCount; i++)
    {
        if (records[i].license == license)
        {
            return i;
        }
    }

    return -1;
}
```

```
// ================= ADMIN LOGIN =================
bool adminLogin()
{
   string username, password;

   cout << "===== ADMIN LOGIN =====" << endl;

   cout << "Enter Username: ";
   cin >> username;

   cout << "Enter Password: ";
   cin >> password;

   if (username == "admin" && password == "1234")
   {
      cout << "\nLogin Successful.\n";
      return true;
   }

   cout << "\nInvalid Username or Password.\n";

   return false;
}

// ================= ADD VEHICLE =================
void addVehicle()
{
   Vehicle v;
```

```
int laneNo;

cout << "\nEnter Lane Number (1-4): ";
cin >> laneNo;

if (laneNo < 1 || laneNo > 4)
{
   cout << "Invalid Lane Number." << endl;
   return;
}

cout << "Enter Vehicle License Number: ";
cin >> v.license;

cout << "Enter Vehicle Type (Emergency/Normal): ";
cin >> v.type;

int index = findRecord(v.license);

if (index != -1 &&
   records[index].status == "SUSPENDED")
{
   cout << "\nVehicle is Suspended." << endl;

   cout << "Suspension Period: "
        << records[index].startDate
        << " to "
```

```
    << records[index].endDate << endl;

cout << "Fine Amount: Rs."
    << records[index].fineAmount << endl;

char choice;

cout << "Has Suspension Period Completed? (Y/N): ";
cin >> choice;

if (choice == 'Y' || choice == 'y')
{
   cout << "Pay Fine Amount: Rs."
        << records[index].fineAmount << endl;

   records[index].fineCount = 0;

   records[index].status = "NORMAL";

   records[index].startDate = "-";

   records[index].endDate = "-";

   records[index].fineAmount = 0;

   saveFile();

   cout << "Vehicle Suspension Removed." << endl;
```

```
        }
        else
        {
            cout << "Vehicle Cannot Enter." << endl;
            return;
        }
    }

    lane[laneNo - 1].push(v);

    cout << "Vehicle Added Successfully." << endl;
}

// ================= ADD VIOLATION =================
void addViolation()
{
    string license;

    cout << "\nEnter Vehicle License Number: ";
    cin >> license;

    int index = findRecord(license);

    if (index == -1)
    {
        records[recordCount].license = license;

        records[recordCount].fineCount = 1;
```

```
    records[recordCount].status = "NORMAL";

    records[recordCount].startDate = "-";

    records[recordCount].endDate = "-";

    records[recordCount].fineAmount = 500;

    cout << "Fine Count: 1" << endl;

    recordCount++;
}
else
{
    records[index].fineCount++;

    cout << "Fine Count: "
         << records[index].fineCount << endl;

    if (records[index].fineCount == 2)
    {
        cout << "\nWarning: One More Violation Will Suspend the Vehicle." << endl;
    }

    if (records[index].fineCount >= 3)
    {
        records[index].status = "SUSPENDED";
```

```
            cout << "\nVehicle Suspended for 3 Months." << endl;

            cout << "Enter Suspension Start Date: ";
            cin >> records[index].startDate;

            cout << "Enter Suspension End Date: ";
            cin >> records[index].endDate;

            cout << "Enter Suspension Fine Amount: ";
            cin >> records[index].fineAmount;
        }
    }

    saveFile();
}

// ================= SHOW STATUS =================
void showStatus()
{
    cout << "\n===== TRAFFIC STATUS =====" << endl;

    for (int i = 0; i < 4; i++)
    {
        cout << "\nLane " << i + 1 << endl;

        cout << "Light: " << lights[i] << endl;
```

```
    cout << "Vehicles: "
         << lane[i].size() << endl;
}

int emergencyCount = 0;

for (int i = 0; i < 4; i++)
{
    queue<Vehicle> temp = lane[i];

    while (!temp.empty())
    {
        if (temp.front().type == "Emergency")
        {
            emergencyCount++;
        }

        temp.pop();
    }
}

cout << "\nEmergency Vehicles Waiting: "
     << emergencyCount << endl;

cout << "\n===== SUSPENDED VEHICLE HISTORY =====" << endl;

bool found = false;
```

```
    for (int i = 0; i < recordCount; i++)
    {
        if (records[i].status == "SUSPENDED")
        {
            found = true;

            cout << "\nLicense Number: "
                 << records[i].license << endl;

            cout << "Fine Count: "
                 << records[i].fineCount << endl;

            cout << "Suspension Period: "
                 << records[i].startDate
                 << " to "
                 << records[i].endDate << endl;

            cout << "Fine Amount: Rs."
                 << records[i].fineAmount << endl;
        }
    }

    if (!found)
    {
        cout << "No Suspended Vehicles." << endl;
    }
}
```

```
// ================= SIMULATE TRAFFIC =================
void simulateTraffic()
{
   // ================= EMERGENCY VEHICLE CHECK =================

   for (int i = 0; i < 4; i++)
   {
      if (!lane[i].empty() &&
         lane[i].front().type == "Emergency")
      {
         lightStack.push(lights[i]);

         for (int j = 0; j < 4; j++)
         {
            lights[j] = "RED";
         }

         lights[i] = "GREEN";

         Vehicle v = lane[i].front();

         int index = findRecord(v.license);

         // ================= SUSPENDED EMERGENCY VEHICLE =================

         if (index != -1 &&
            records[index].status == "SUSPENDED")
         {
```

```
cout << "\nVehicle is Suspended." << endl;

cout << "Suspension Period: "
    << records[index].startDate
    << " to "
    << records[index].endDate << endl;

cout << "Fine Amount: Rs."
    << records[index].fineAmount << endl;

char choice;

cout << "Has Suspension Period Completed? (Y/N): ";
cin >> choice;

// ===== IF COMPLETED =====
if (choice == 'Y' || choice == 'y')
{
   cout << "Pay Fine Amount: Rs."
        << records[index].fineAmount << endl;

   records[index].fineCount = 0;

   records[index].status = "NORMAL";

   records[index].startDate = "-";

   records[index].endDate = "-";
```

```
            records[index].fineAmount = 0;

            saveFile();

            cout << "Vehicle Suspension Removed." << endl;

            cout << "\nEmergency Vehicle Passed: "
                 << v.license << endl;
        }

        // ===== IF NOT COMPLETED =====
        else
        {
            cout << "\nSuspended Emergency Vehicle Allowed Due to Emergency Priority."
<< endl;

            records[index].fineAmount += 1000;

            cout << "Additional Emergency Penalty Added." << endl;

            cout << "Updated Fine Amount: Rs."
                 << records[index].fineAmount << endl;

            cout << "Fine Count: "
                 << records[index].fineCount << endl;

            saveFile();
```

```
            cout << "\nEmergency Vehicle Passed: "
                 << v.license << endl;
        }
    }

    // ================= NORMAL EMERGENCY VEHICLE =================

    else
    {
        cout << "\nEmergency Vehicle Passed: "
             << v.license << endl;
    }

    lane[i].pop();

    lights[i] = lightStack.top();

    lightStack.pop();

    return;
  }
}

// ================= NORMAL TRAFFIC SIMULATION =================

int laneNo;
```

```
cout << "\nEnter Lane Number to Simulate (1-4): ";
cin >> laneNo;

if (laneNo < 1 || laneNo > 4)
{
   cout << "Invalid Lane Number." << endl;
   return;
}

laneNo--;

if (lane[laneNo].empty())
{
   cout << "No Vehicles in Selected Lane." << endl;
   return;
}

if (lights[laneNo] == "RED")
{
   lightStack.push(lights[laneNo]);

   for (int i = 0; i < 4; i++)
   {
      lights[i] = "RED";
   }

   lights[laneNo] = "GREEN";
}
```

```
Vehicle v = lane[laneNo].front();

int index = findRecord(v.license);

// ================= SUSPENDED NORMAL VEHICLE =================

if (index != -1 &&
   records[index].status == "SUSPENDED")
{
   cout << "\nVehicle is Suspended." << endl;

   cout << "Suspension Period: "
        << records[index].startDate
        << " to "
        << records[index].endDate << endl;

   cout << "Fine Amount: Rs."
        << records[index].fineAmount << endl;

   char choice;

   cout << "Has Suspension Period Completed? (Y/N): ";
   cin >> choice;

   if (choice == 'Y' || choice == 'y')
   {
      cout << "Pay Fine Amount: Rs."
```

```
            << records[index].fineAmount << endl;

        records[index].fineCount = 0;

        records[index].status = "NORMAL";

        records[index].startDate = "-";

        records[index].endDate = "-";

        records[index].fineAmount = 0;

        saveFile();

        cout << "Vehicle Suspension Removed." << endl;
    }
    else
    {
        cout << "Suspended Vehicle Cannot Pass." << endl;
        return;
    }
}

cout << "\nSignal: GREEN" << endl;

lane[laneNo].pop();

cout << "Vehicle Passed: "
```

```
        << v.license << endl;

    cout << "Signal: YELLOW" << endl;

    lights[laneNo] = "YELLOW";

    cout << "Signal: RED" << endl;

    lights[laneNo] = "RED";
}

// ================= MAIN FUNCTION =================
int main()
{
    if (!adminLogin())
    {
        return 0;
    }

    loadFile();

    int choice;

    do
    {
        cout << "\n===== TRAFFIC MANAGEMENT SYSTEM =====" << endl;

        cout << "1. Add Vehicle" << endl;
```

```
cout << "2. Add Traffic Violation" << endl;

cout << "3. Show Traffic Status" << endl;

cout << "4. Simulate Traffic" << endl;

cout << "5. Exit" << endl;

cout << "Enter Your Choice: ";
cin >> choice;

switch (choice)
{
   case 1:
      addVehicle();
      break;

   case 2:
      addViolation();
      break;

   case 3:
      showStatus();
      break;

   case 4:
      simulateTraffic();
```

```
            break;

        case 5:
            saveFile();

            cout << "\nProgram Ended." << endl;
            break;

        default:
            cout << "\nInvalid Choice." << endl;
      }

   } while (choice != 5);

   return 0;
}
```